GhostSnap

Replicazione di interazioni DesignScape con Fabric.js

Francesco Simbola

Anno Accademico 2024/2025

Sommario

Il presente progetto, denominato **GhostSnap**, esplora tecniche avanzate di interazione utente in ambienti di editing grafico web. Ispirato dal paper *DesignScape* [1], il sistema implementa un meccanismo di allineamento assistito basato su suggerimenti visivi temporizzati ("Ghosts"). L'applicazione è stata sviluppata utilizzando **React** e la versione 6 (beta) della libreria **Fabric.js**, affrontando sfide tecniche legate alla gestione di eventi asincroni, performance di rendering e gestione dello stato storico (Undo/Redo).

Indice

1 Introduzione **3**
1.1 Il Paradigma Invisible Drag 3

2 Stato dell'Arte **3**

3 Obiettivi e Differenze con DesignScape **3**
3.1 Generativo vs Deterministico 3
3.2 Interazione vs Ottimizzazione 4

4 Architettura e Implementazione **4**
4.1 Gestione dello Stato 4

5 Analisi del Workflow Utente **4**
5.1 Interfaccia Iniziale e Griglia 4
5.2 Creazione e Manipolazione Oggetti 6
5.3 Spostamento Vincolato (Grid Snap) 7
5.4 Spostamento Libero e Gestione Immagini 8

6 Sfide Tecniche e Soluzioni **10**
6.1 Il Problema della "Scia" (The Trail Effect) 10
6.2 Race Conditions e "Oggetti Zombie" 11
6.3 Undo System e Asincronia in Fabric v6 11

7 Conclusioni e Sviluppi Futuri **11**

1 Introduzione

L'obiettivo del progetto è migliorare l'esperienza di allineamento degli oggetti in un canvas grafico. Nei software tradizionali di authoring (come PowerPoint o Figma), le guide di allineamento ("snap lines") appaiono immediatamente durante il trascinamento. Questo approccio, sebbene funzionale, può generare rumore visivo ("visual clutter") quando molti oggetti sono presenti sulla scena.

In *GhostSnap*, abbiamo implementato un paradigma diverso per ridurre il carico cognitivo: il suggerimento di allineamento appare solo quando l'utente manifesta un'intenzione di posizionamento esplicita, fermandosi per un breve lasso di tempo.

1.1 Il Paradigma Invisible Drag

L'interazione è stata progettata in tre fasi distinte per massimizzare la chiarezza:

1. **Trascinamento (Invisible Drag)**: Durante il movimento, l'oggetto originale rimane statico (semi-trasparente) al punto di partenza. Il cursore si muove "vuoto", permettendo una visione chiara dell'area di destinazione.

2. **Attesa (Debounce Timer)**: Se l'utente ferma il cursore vicino al centro del canvas per 60ms (configurabile), il sistema calcola la posizione di allineamento e mostra un "Fantasma" (Ghost) blu.

3. **Rilascio (Snap)**: Al rilascio del mouse, se il Fantasma è visibile, l'oggetto reale viene "teletrasportato" nella posizione suggerita.

2 Stato dell'Arte

La generazione assistita di layout e l'allineamento automatico sono temi centrali nella ricerca HCI (Human-Computer Interaction).

Strumenti commerciali come **Microsoft PowerPoint Designer** [3] e **Canva** [2] utilizzano sistemi template-based o euristici per suggerire layout completi all'utente. Tuttavia, questi sistemi operano spesso come "scatole nere", rigenerando l'intera diapositiva e offrendo poco controllo sulle micro-interazioni di singolo oggetto.

Al contrario, i sistemi basati su vincoli (Constraint Solvers) come **Cassowary** [4], integrati nelle moderne interfacce di design (es. Figma, Sketch), offrono un controllo granulare tramite guide magnetiche immediate. Il limite di questo approccio è il sovraccarico visivo: l'utente viene bombardato da linee guida anche durante movimenti rapidi di esplorazione.

DesignScape [1] si posiziona tra questi due estremi, proponendo suggerimenti interattivi che migliorano il layout esistente. *GhostSnap* riprende questa filosofia, focalizzandosi specificamente sulla riduzione del disturbo visivo durante l'atto di trascinamento.

3 Obiettivi e Differenze con DesignScape

Sebbene questo progetto tragga forte ispirazione dal lavoro di O'Donovan et al. [1], esistono differenze sostanziali negli obiettivi e nell'implementazione tecnica.

3.1 Generativo vs Deterministico

Il sistema originale **DesignScape** utilizza complessi algoritmi di ottimizzazione stocastica (come il *Simulated Annealing* e il *Parallel Tempering*) per generare suggerimenti di layout. Il sistema esplora migliaia di possibili configurazioni per trovarne una ottimale basata su una "funzione di energia".

In **GhostSnap**, abbiamo scelto un approccio differente:

- **DesignScape** usava un motore probabilistico per "inventare" nuovi layout.

- **Noi abbiamo scelto** un approccio **deterministico e reattivo**. Il nostro sistema non cerca di ridisegnare la slide, ma assiste l'utente nell'azione atomica di posizionamento.

Questa scelta è motivata dalla volontà di garantire un feedback istantaneo (60fps) nel browser, senza la latenza introdotta da calcoli di ottimizzazione complessi server-side o background.

3.2 Interazione vs Ottimizzazione

Mentre il paper originale si concentra sull'ottimizzazione globale della slide (es. bilanciamento, simmetria totale), il nostro progetto isola il contributo dell'interazione **Invisible Drag**. L'obiettivo non è la perfezione estetica automatica, ma la pulizia dell'interazione: fornire l'aiuto giusto (il fantasma) solo al momento giusto (la pausa).

4 Architettura e Implementazione

Il progetto è sviluppato come Single Page Application (SPA) utilizzando **React**. La gestione della grafica vettoriale è affidata a **Fabric.js v6**.

4.1 Gestione dello Stato

Per evitare re-render eccessivi di React che avrebbero compromesso la fluidità del canvas, la logica è gestita tramite `useRef`. Questo permette di mantenere riferimenti mutabili agli oggetti Fabric senza innescare il ciclo di aggiornamento del DOM virtuale.

- `ghostObject`: Riferimento all'oggetto "fantasma" blu.
- `staticPlaceholder`: La copia statica che rimane al punto di partenza.
- `dragTimer`: Il timer per gestire il ritardo dell'apparizione (Debouncing).

5 Analisi del Workflow Utente

In questa sezione analizziamo il funzionamento dell'applicazione attraverso una serie di schermate che illustrano le diverse modalità di interazione.

5.1 Interfaccia Iniziale e Griglia

All'avvio, l'applicazione presenta una toolbar e l'area di lavoro (Canvas). Di default, è attiva la visualizzazione della "Griglia", rappresentata da due assi cartesiani centrali (**Figura 1**).

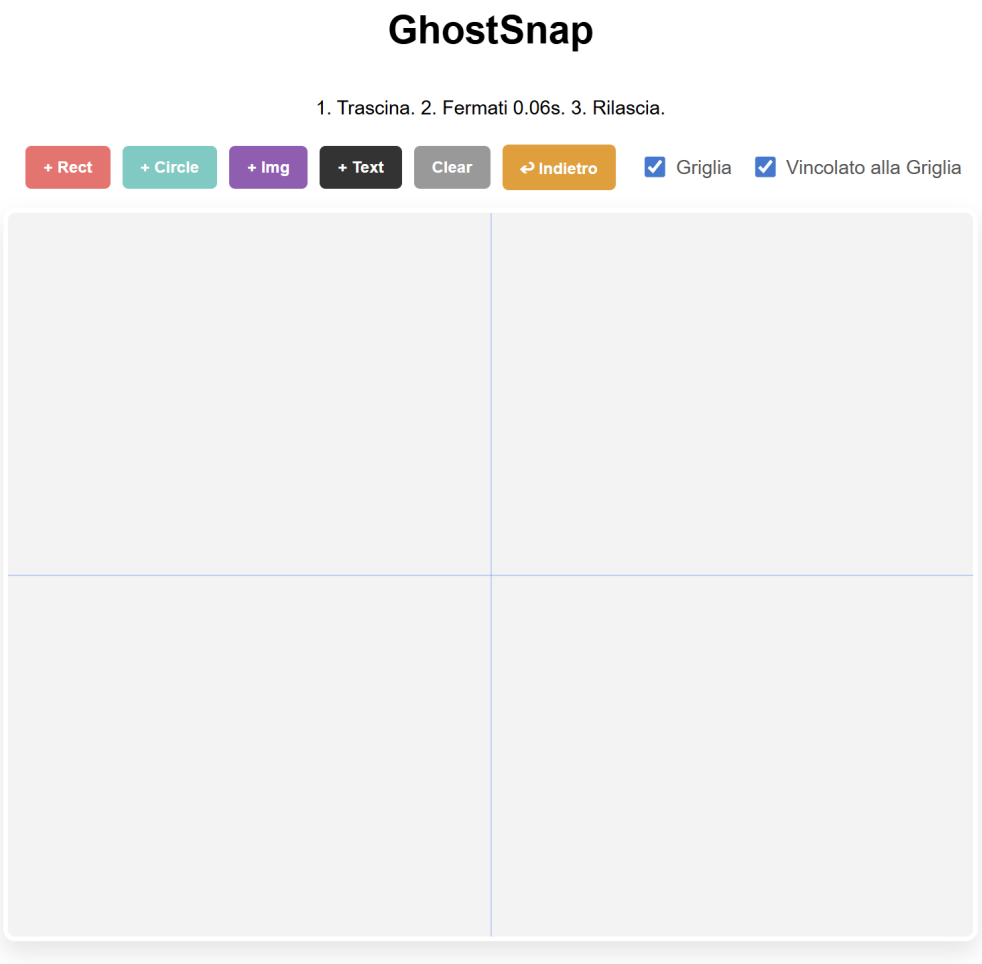


Figura 1: Schermata iniziale con griglia attiva e toolbar comandi.

L'utente ha la possibilità di disattivare la griglia visiva tramite la checkbox apposita (**Figura 2**), modificando lo stato React `showGrid`.

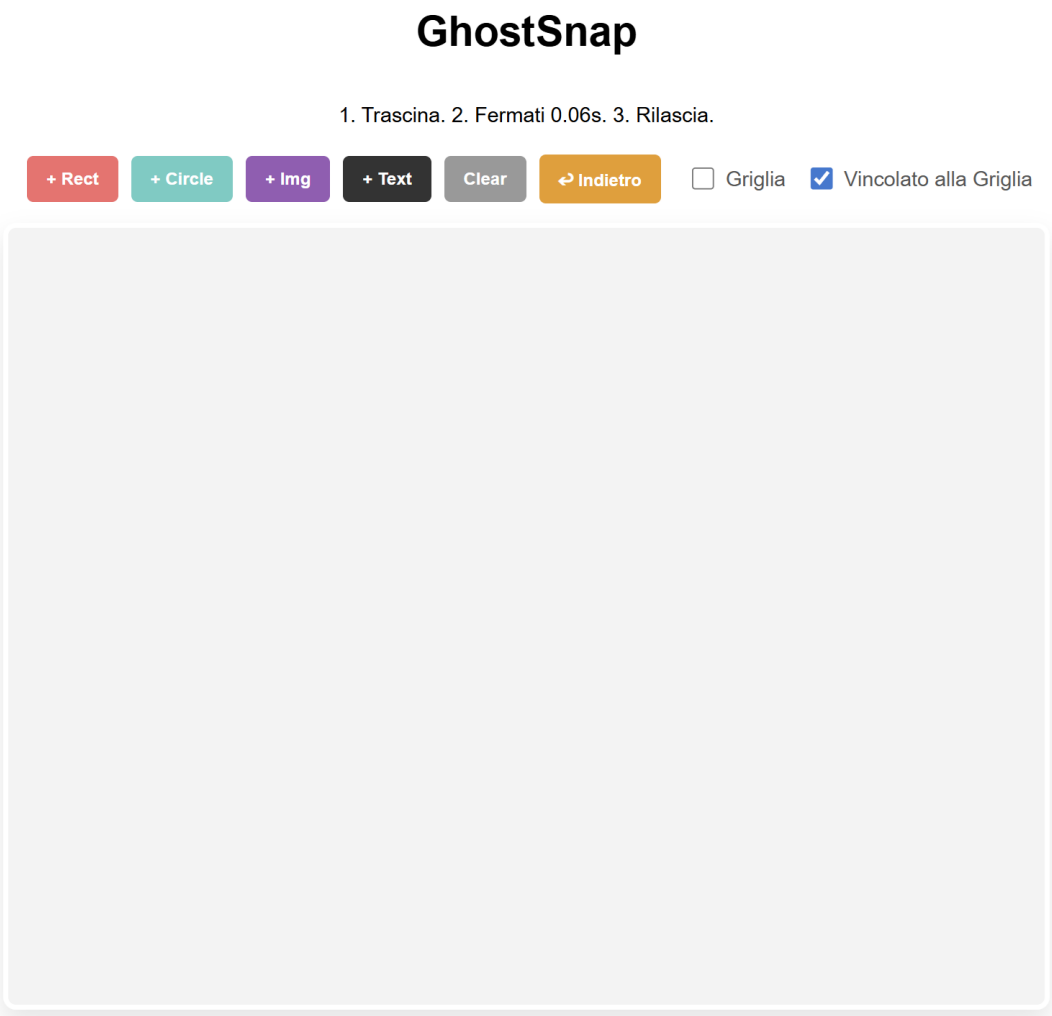


Figura 2: Visualizzazione pulita con griglia disattivata.

5.2 Creazione e Manipolazione Oggetti

Cliccando sui pulsanti nella toolbar, viene istanziato un nuovo oggetto Fabric.js. L'oggetto appare al centro e presenta i controlli di manipolazione (**Figura 3**).

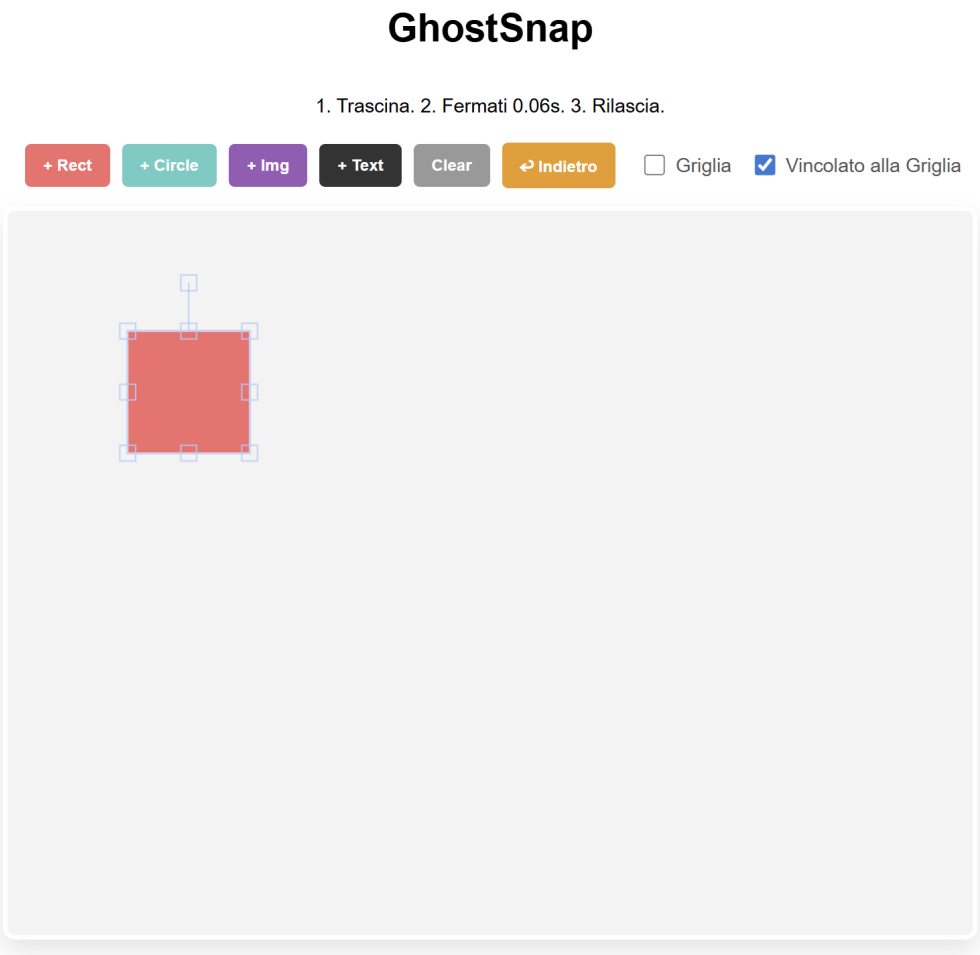


Figura 3: Aggiunta di un rettangolo. Sono visibili i controlli di selezione.

Il codice sottostante gestisce l'aggiunta, assicurandosi di impostare l'oggetto come "attivo":

```
const addRect = () => {
  if(!fabricCanvas.current) return;
  const rect = new fabric.Rect({
    left: 150, top: 150, fill: '#ff6b6b', width: 100, height: 100,
    originX: 'center', originY: 'center', cornerRadius: 8
  });
  fabricCanvas.current.add(rect);
  fabricCanvas.current.setActiveObject(rect);
};
```

Listing 1: Funzione di aggiunta rettangolo

5.3 Spostamento Vincolato (Grid Snap)

Quando la checkbox "Vincolato alla Griglia" è attiva, il fantasma appare **solamente** se l'oggetto si trova in prossimità degli assi centrali. In **Figura 4**, il quadrato rosso originale (statico) rimane a sinistra, mentre il fantasma blu appare centrato sulla griglia, poiché l'utente ha fermato il cursore entro la soglia di snap.

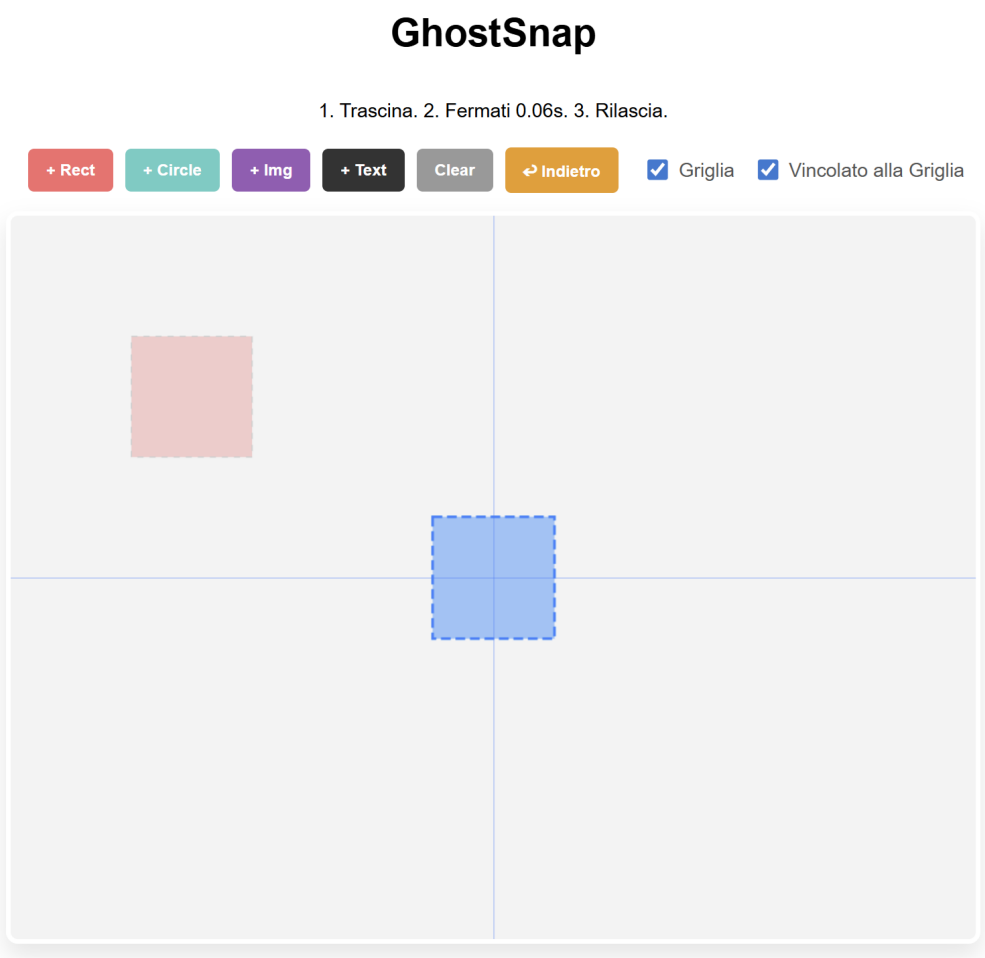


Figura 4: Il fantasma blu appare al centro (snap magnetico) mentre l'originale resta fermo.

5.4 Spostamento Libero e Gestione Immagini

Disattivando la spunta "Vincolato alla Griglia", si abilita la modalità libera. Il fantasma funge da conferma visiva della posizione attuale.

La **Figura 5** mostra l'interazione con un'immagine (Stormtrooper): il fantasma rettangolare (bounding box) appare esattamente dove si trova il cursore.

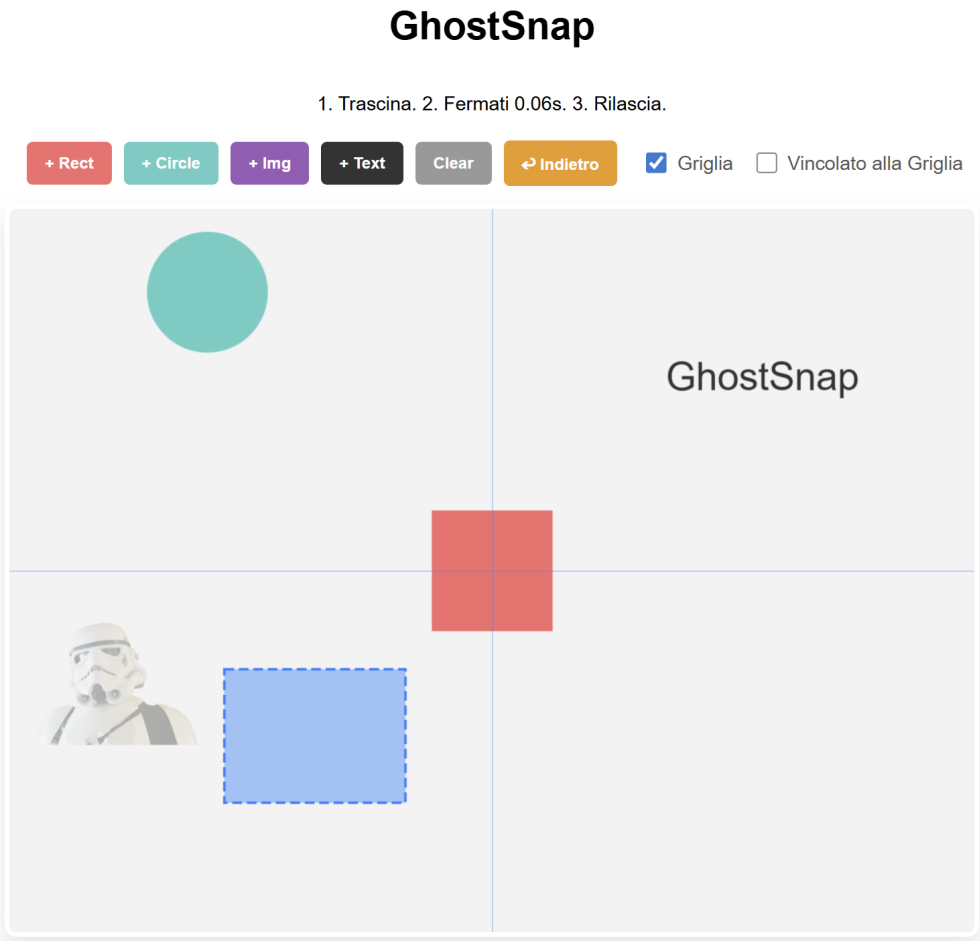


Figura 5: Modalità libera: il fantasma appare nella posizione corrente del cursore.

Al rilascio del mouse, l'oggetto viene spostato nella posizione suggerita (**Figura 6**).

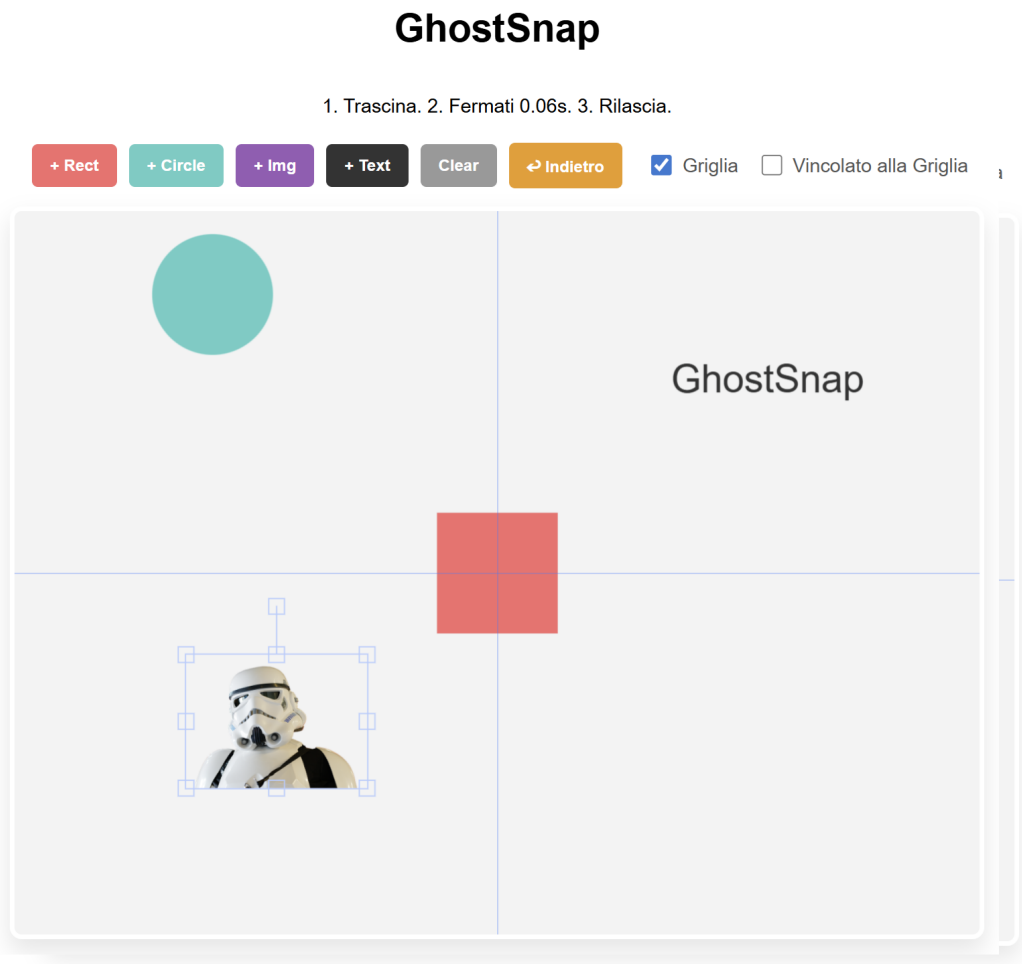


Figura 6: Spostamento completato.

Notiamo che per l'immagine, il sistema genera un fantasma rettangolare corrispondente all'ingombro dell'immagine stessa, grazie alla logica polimorfica:

```
// Se non e' un cerchio, usa un rettangolo (gestisce Rect, Text e Image)
else {
    ghostShape = new fabric.Rect({
        width: obj.getScaledWidth(),
        height: obj.getScaledHeight(),
        // ... stili del fantasma
    });
}
```

Listing 2: Gestione polimorfica del fantasma

6 Sfide Tecniche e Soluzioni

6.1 Il Problema della "Scia" (The Trail Effect)

Inizialmente, la creazione del fantasma avveniva ad ogni evento `object:moving`. Poiché Fabric.js v6 gestisce la clonazione in modo asincrono, un movimento rapido generava centinaia di clonazioni in ritardo. **Soluzione:** Abbiamo adottato una strategia a *Single Instance*. Il fantasma viene creato una sola volta e successivamente riposizionato, azzerando il carico computazionale.

6.2 Race Conditions e "Oggetti Zombie"

Se l'utente cliccava e rilasciava molto velocemente, l'evento `mouse:up` scattava prima che la Promise di clonazione del `mouse:down` fosse completata, lasciando oggetti "orfani". **Soluzione:** Implementazione di una funzione di *Garbage Collection* aggressiva (`clearAllPlaceholders`) invocata a inizio e fine interazione.

6.3 Undo System e Asincronia in Fabric v6

Il metodo `loadFromJSON` in Fabric v6 è asincrono. L'uso errato causava il rendering di un canvas bianco. **Soluzione:** Riscrittura del sistema Undo per attendere la risoluzione della Promise.

```
fabricCanvas.current.loadFromJSON(previousState).then((canvas) => {
    canvas.renderAll(); // Rendering forzato post-Promise
    ghostObject.current = null;
});
```

Listing 3: Gestione corretta dell'Undo asincrono

7 Conclusioni e Sviluppi Futuri

Il progetto *GhostSnap* dimostra l'efficacia dell'*Invisible Drag* nel ridurre il carico visivo. A differenza dell'approccio generativo di DesignScape, la nostra implementazione deterministica garantisce performance elevate in ambiente web standard. Sviluppi futuri potrebbero includere il supporto per linee guida multiple e il ridimensionamento assistito.

Riferimenti bibliografici

[1] Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. 2015. *DesignScape: Design with Interactive Layout Suggestions*. In Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15). ACM.

[2] Canva. *Online Graphic Design Platform*. URL: https://www.canva.com.

[3] Microsoft. *PowerPoint Designer*. URL: https://support.microsoft.com/en-us/office/create-professional-slide-layouts-with-designer.

[4] Greg J. Badros, Alan Borning, and Philip J. Stuckey. 2001. *The Cassowary Linear Arithmetic Constraint Solving Algorithm*. ACM Trans. Comput.-Hum. Interact.